Preprint
CC BY 4.0

Deploying LLM Inference on a Repurposed UMA APU: Transferable Lessons from a Vulkan-Only, 16 GB Edge Platform

Artur Andrzejczak
(Independent Researcher, Łódź, Poland
https://orcid.org/0009-0001-9169-1622,
andrzejczak.artur@gmail.com)

Abstract: Cost-driven interest in running large language models (LLMs) on non-mainstream silicon outpaces the maturity of the surrounding software stacks. This paper uses one such platform, the AMD BC-250 (a repurposed cryptocurrency-mining board with a GFX1013 "Cyan Skillfish" accelerated processing unit (APU), 16 GB of unified GDDR6 memory, and a Vulkan-only GPU compute path), as a single-board testbed. Three observations not specific to this board are reported: two benchmark-validity cautions and one controlled empirical result. The cautions are (i) a silent prompt-truncation failure mode in the Ollama runtime that affects filled-context benchmarking unless the API's `prompt_eval_count` field is verified per request, and (ii) the operational distinction between context *allocation* and context *utilisation*, which can produce headline "128K context" numbers backed by a much shorter prompt. The empirical result is (iii) that on hardware lacking matrix accelerators, sparse mixture-of-experts (MoE) generation throughput appears to track the active-parameter count rather than the total. The practical edge over a deployable dense model is modest (a 35 B-total / 3 B-active MoE leads the 14 B dense models by ≈14–25 % at short context); the larger ≈6× separation reported here is against a same-size dense control held at the same ≈2.5-bit quantisation, a configuration chosen to isolate the variable rather than one intended for deployment. Both readings are consistent with a roofline measurement that places decode left of the compute ridge, in a regime bounded by memory bandwidth relative to compute. The evidence rests on a 31-model core cohort (3–35 B parameters), with six further follow-up reference builds added for targeted sub-studies, covering generation speed (statistically validated; within-cell coefficient of variation (CV) $\leq$ 0.4 % across the canonical n=3 cells), filled-context scaling with 80 % real-token payloads, output quality across five task types, cold-start latency, and clock and thermal telemetry. A reproducible deployment recipe is documented as the enabling infrastructure for these measurements: a kernel TTM (Translation Table Manager) `pages_limit` adjustment plus an independently FP-validated reuse of a community 40-CU core unlock.

Keywords: edge AI; large language models; mixture of experts; unified memory architecture; Vulkan inference; benchmarking methodology; reproducibility; runtime correctness; GGUF quantisation
Categories: D.4.8 Performance, I.2.7 Natural Language Processing
DOI: 10.5281/zenodo.19476016

1 Introduction

The cost gap between data-centre LLM inference and the consumer or edge tier is large, persistent, and increasingly consequential for privacy, latency, and data-sovereignty

reasons [Chen and Ran 2019]. Closing it has driven interest in unified-memory architectures (UMA), repurposed accelerators, and non-CUDA software stacks. These platforms inherit a smaller share of the maturity invested in the NVIDIA/CUDA mainline, and that asymmetry shows up not only in raw performance but in the trustworthiness of the surrounding tooling: silent runtime errors, configuration knobs whose effect is workload-dependent, and benchmark protocols whose validity is conditional on stack-specific behaviour. The community work-arounds that fill the gap are typically published as GitHub issues, vendor-internal patches, and single-author measurement reports rather than as peer-reviewed artefacts.

This paper studies one such platform, the AMD BC-250: a board originally produced in volume for cryptocurrency mining and now available on the secondary market at a fraction of the cost of discrete GPUs of comparable memory capacity, used here as a single-board testbed. Each board integrates a Zen 2 CPU (6 cores / 12 threads) with a GFX1013 "Cyan Skillfish" GPU and a single 16 GB GDDR6 pool shared between CPU and GPU through a 256-bit bus. GFX1013 is a GFX10.1-era part incorporating select RDNA 2 features, referred to as "RDNA 1.5" in vendor-adjacent developer-community materials [Larabel 2025, LLVM 2026]. GFX1013 is not in AMD's ROCm support list; in the tested configuration ROCm userspace failed at runtime, and OpenCL through Mesa's rusticl was likewise unusable. Vulkan via the Mesa RADV driver was the only GPU compute path verified as functional, and even this path required kernel-level adjustments before models above $\approx$7 B parameters could run.

Headline findings. Three observations from this work travel beyond the specific board: two are methodological cautions about benchmark validity (findings 1–2), and the third (finding 3) is an empirical result established by a controlled, single-variable experiment and read through a roofline measurement. They are listed in order of generality.

1. **Silent prompt truncation in Ollama.** The Ollama runtime, a widely used HTTP layer over llama.cpp [ggml-org 2026, Ollama 2026], can silently truncate prompts above a model's native context window: the API returns HTTP 200 with a reduced `prompt_eval_count` and no error indication. A filled-context benchmark that does not verify `prompt_eval_count` can report headline numbers that do not correspond to the prompt actually processed (Section 4.3). This failure mode is a property of the runtime, not of the BC-250.

2. **Allocation $\neq$ utilisation in filled-context benchmarks.** On this 16 GB UMA, `num_ctx` $=$ 131 072 (the 128K tier) can be *allocated* for every model tested; *utilising* 128K with real tokens timed out under the original 20-minute budget for larger models (MoE-35B, 14B class), dominated by prefill; lighter models completed within budget ($\approx$15.5 and 2.4 min time to first token (TTFT) for `qwen3.5:9b` and `granite-4.0-h-tiny`, directly measured; Section 4.3). Reports of large supported contexts that do not separate the two are ambiguous, and the distinction is independent of the platform.

3. **Sparse MoE on scalar hardware.** On a GPU lacking dedicated matrix accelerators, where compute dispatches through scalar ALUs and throughput is consistent with a memory-bandwidth-bounded regime (Section 7.2), the 35 B-total / 3 B-active Qwen3.5-35B-A3B MoE reaches 25.1 tok/s at 4K context, ahead of the tested 14 B dense models (20–22 tok/s) by a modest $\approx$14–25 %. The result is consistent with decode-time throughput on such hardware tracking the active-parameter count rather than the total [Fedus et al. 2022], and it is the one finding resting on a controlled

single-variable comparison: with quantisation and runtime held fixed, a same-size dense comparator and the 3 B-active MoEs are several-fold apart (Sections 5.1, 7.2, Figure 1). This advantage is *not* confined to short context: the MoE runs to 64K filled context at 16.6 tok/s (Section 5.3).

Enabling infrastructure: deployment recipe and CU unlock. A working stack on the BC-250 requires two adjustments below the runtime layer. First, the kernel's TTM `pages_limit` must be raised before models above ≈7 B parameters can run on the GPU; without this, 14 B-class models load but produce HTTP 500 during inference at the point the runtime extends the key-value (KV) cache allocation past the TTM ceiling. Second, the GFX1013 die ships with 40 physical Compute Units of which the factory firmware harvest mask exposes only 24; a community kernel-module modification by S. Duggan [Duggan 2026] re-enables the remaining 16 CUs without firmware modification. Both adjustments are reused work, attributed in Sections 3.3–3.4; this paper's contribution at this layer is an independent FP-correctness sanity check of the re-enabled silicon (Section 3.4) and a paired-arm controlled re-run of the throughput effect on the same board (Section 5.7), both reported as *enabling* the higher-level findings rather than as the headline.

Contributions. Beyond finding 3's controlled comparison (Sections 5.1, 7.2, Figure 1), the paper contributes a 31-model characterisation (3–35 B parameters) on a non-NVIDIA, non-ROCm, Vulkan-only stack, together with reproducibility artefacts: the `prompt_eval_count` truncation-detection recipe, an independent FP32 sanity check and paired-arm CU-unlock re-run, the consolidated deployment recipe, and a threats-to-validity consolidation of the single-device, single-driver, and single-firmware limitations (Section 6).

2 Related Work

The majority of published LLM-inference research targets discrete GPUs with dedicated VRAM. The llama.cpp project [ggml-org 2026] pioneered quantised GGUF execution on CPUs and on mixed CPU/GPU configurations; Ollama [Ollama 2026] provides an HTTP API layer above it. Both are open-source projects without peer-reviewed artefacts; they are cited here as the runtime stack under test, not as evidentiary sources. Published benchmarks overwhelmingly focus on NVIDIA/CUDA or AMD/ROCm hardware [Zhou et al. 2024]; Vulkan-only inference platforms appear underrepresented in the peer-reviewed literature.

Recent UMA work, most prominently on Apple Silicon [Benazir and Lin 2025], has shown that shared CPU–GPU memory pools can sustain practical LLM-inference throughput. The BC-250's UMA is analogous in concept but differs in implementation: GDDR6 instead of LPDDR5, no dedicated matrix-accelerator units, and Vulkan instead of Metal as the compute API. Post-training quantisation techniques (GPTQ [Frantar et al. 2023], AWQ [Lin et al. 2024], GGUF IQ2 variants) compress models by 4–8× with controlled quality degradation. The combination of ≈2.5-bit quantisation with Vulkan inference on non-standard GPU silicon has not, to the author's knowledge, been examined in prior peer-reviewed work, and the empirical characterisation in this paper fills that gap on one such silicon.

Mixture-of-Experts (MoE) architectures [Fedus et al. 2022], for which efficient inference at scale has motivated dedicated serving studies [Pope et al. 2023], activate only

a subset of their parameters per token. On hardware lacking dedicated matrix cores, where computation is dispatched through scalar ALUs and per-token decode is consistent with a memory-bandwidth-bounded regime, the throughput advantage of sparse activation is expected to be more pronounced; this paper contributes empirical data relevant to that expectation.

The 31-model core cohort carried through Phases 1–5 and the basic-task quality and context tables (Tables 3, 8, 5) spans the Qwen 2.5, Qwen 3 and Qwen 3.5 [Qwen Team 2024, Qwen Team 2025a, Qwen Team 2026a], Llama 3 [Grattafiori et al. 2024], Gemma 2 [Gemma Team 2024] and Gemma 3 [Gemma Team 2025], Phi-4 [Abdin et al. 2024], DeepSeek-R1 [DeepSeek-AI 2025], IBM Granite 3.3, Mistral, GLM-4 and Seed-Coder families. A further six reference models, added for specific follow-up studies, are *not* part of this basic-task cohort and are reported only where named: two sparse MoEs for the cross-family comparison (Section 5.1) and 40-CU re-run, namely the Qwen 3.6 35B-A3B MoE [Qwen Team 2026b] (a distinct build from the cohort's `qwen3.5-35b-a3b` at the same 35 B/3 B split) and the community Gemma-4 26B-A4B fork `gemma4-latest` (≈4 B active, Q3, Ollama-served because tip-of-tree llama.cpp rejects its `gemma4` architecture); the IBM Granite 4.0 Mamba–MoE hybrid `granite-4.0-h-tiny` [Granite Team 2025] (Sections 5.4.4, 5.4.3); the OpenAI `gpt-oss-20b` MXFP4 MoE [OpenAI 2025] (Section 5.6.1); and the dense reasoning/instruction pair `QwQ-32B` [Qwen Team 2025b] and `Mistral-Small-3.2-24B` [Mistral AI 2025] (Section 5.3.4). Where model documentation appears only on a vendor blog, model card, or community fork (Qwen 3, Qwen 3.5, Qwen 3.6, Granite 4.0, `gpt-oss`, GLM-4, Seed-Coder, QwQ-32B, Mistral Small 3.2, and the Gemma-4 community fork), citations or inline provenance notes are flagged as non-peer-reviewed and used only to identify weights and quantisation; no quantitative claim of this paper rests on those sources.

3 Hardware and Software Stack

3.1 AMD BC-250 Specifications

The BC-250 board uses an AMD Cyan Skillfish accelerated processing unit (APU) (Table 1). CPU and GPU share a single 16 GB GDDR6 pool through a 256-bit bus. The BIOS reserves 512 MB as a framebuffer; the remainder is allocated to the Graphics Translation Table (GTT) once kernel limits are raised. In this UMA configuration, "100 % GPU offload" means model weights and KV cache reside in GTT pages that the GPU reads directly; no PCIe copy is involved.

The GFX1013 die ships with 40 physical Compute Units (CUs), of which the factory firmware harvest mask leaves only 24 active by default. This paper reports speed and quality numbers for the stock 24-CU configuration throughout Sections 5.1–5.6, and reports delta measurements after the community 40-CU unlock (Section 3.4) separately in Sections 5.7 and 5.8.

The board shipped with a minimal factory BIOS designed for rack-mount mining. A community-patched BIOS [BC-250 Docs 2026] adds standard UEFI features including NVMe boot and fan control; the tested board runs this patched BIOS (AMI P3.00).

3.2 Vulkan as the Sole GPU Compute Path

GFX1013 is listed in LLVM's AMDGPU target table as supporting `rocm-amdhsa` [LLVM 2026]; however, in the tested configuration ROCm userspace

Table 1: AMD BC-250 hardware specifications. "Active CUs" lists the stock factory-harvest value first and the count obtainable via the community unlock (Section 3.4) in parentheses. The board-vendor documentation referenced is a community-maintained wiki [BC-250 Docs 2026], not a manufacturer datasheet.

Component	Details
CPU	Zen 2, 6 cores / 12 threads
GPU	Cyan Skillfish (GFX1013, "RDNA 1.5")
	40 physical CUs total, 24 active stock (40 after unlock)
	1536 SPs stock (2560 SPs after unlock)
Memory	16 GB GDDR6 unified, 256-bit bus
GTT (tuned)	16 GiB via `ttm.pages_limit`; ≈13 GiB model+KV usable
Vulkan visible	16.5 GiB after tuning (two logical heaps)
OS	Fedora 43, kernel 6.18.9-200.fc43.x86_64
TDP (nameplate)	220 W [BC-250 Docs 2026]

components were not usable on GFX1013, and attempts resulted in a `rocblas_abort()` failure (Table 2). OpenCL via Mesa's rusticl also did not work (Mesa 25.3.4, Fedora 43). Vulkan through the Mesa RADV driver [Mesa 2026] was the only GPU compute path verified as usable in the tested environment. All measurements reported in this paper run on this Vulkan path, served either through the Ollama HTTP front-end or directly through the llama.cpp `llama-completion` driver (the serving path is stated with each number; Section 4); the canonical throughput and retrieval numbers in Tables 3–7 and 10–12 are from a co-validated Ollama 0.20.0 / llama.cpp b9265 stack (Section 4).

Table 2: Compute-framework availability on the BC-250 (Fedora 43, kernel 6.18.9, Mesa 25.3.4).

Framework	Status	Notes
amdgpu driver	Works	Auto-detected
Vulkan (RADV)	Works	Mesa 25.3.4, sole compute path
ROCm / HIP	Fails	GFX1013 not in supported GPU list
OpenCL (rusticl)	Fails	Not usable in tested env.

3.3 Kernel Memory Tuning

Two kernel-level memory limits required adjustment before models above ≈7 B parameters could run on the GPU.

GTT default cap. The `amdgpu` kernel driver defaults to allocating 50 % of physical RAM as GTT (≈7.4 GiB on a 16 GB system). The legacy fix (`amdgpu.gttsize`) is deprecated in modern kernels [Kernel AMDGPU Params 2026].

TTM `pages_limit`. The kernel's TTM (Translation Table Manager) memory manager independently caps GPU allocations at ≈7.4 GiB. Without this adjustment, 14 B-class models load but produce HTTP 500 errors during inference at the

point the runtime extends the KV-cache allocation past the TTM ceiling. Setting `ttm.pages_limit=4194304` (16 GiB in 4 KiB pages) lifts both limits, and the value holds: `pages_limit` reads back as 4 194 304 and a 12.6 GiB model loads with its full GTT footprint.

The setting is applied from three independent sources (kernel cmdline, `modprobe.d`, and an `/etc/tmpfiles.d` entry) and verified to read back as 4 194 304 across reboots. With `pages_limit` at 16 GiB the practical model+KV working-set envelope is ≈13 GiB, bounded by the 16 GB *physical* pool (weights, KV cache, compute buffers and the OS) rather than by `pages_limit`; the in-tree TTM source documents the parameter [Kernel TTM 2025].

After tuning, Vulkan reports 16.5 GiB across two logical heaps (a 5.5 GiB host-visible BAR-mapped heap and an 11 GiB device-local heap), backed by the same physical 16 GB pool; how that split behaves under `GGML_VK_PREFER_HOST_MEMORY` heap-placement controls is characterised in the supplementary material [Andrzejczak 2026].

3.4 Compute-Unit Activation (40-CU Unlock)

Attribution. The community kernel-module modification described in this subsection is the work of S. Duggan, published as an out-of-tree AMDGPU kernel patch [Duggan 2026] (a non-peer-reviewed open-source repository). This paper does not claim authorship of the modification. The contribution at this layer is the independent floating-point correctness sanity check described at the end of this subsection and the paired-arm throughput re-run reported in Section 5.7.

The GFX1013 die on the BC-250 has 40 physical Compute Units, but the board's factory harvest mask programs the chip to expose only 24 of them (6 of the 10 CUs in each of the four SE×SH shader arrays). The Duggan modification re-enables the remaining 16 CUs by writing two GFX10 configuration registers during the driver's per-array initialisation loop in `gfx_v10_0_get_cu_info()`:

- `CC_GC_SHADER_ARRAY_CONFIG`: `0xfff80000` → `0xffe00000` (driver-visible enumeration mask).
- `SPI_PG_ENABLE_STATIC_WGP_MASK`: `0x00000007` → `0x0000001f` (dispatch gate determining which Work-Group Processors the Shader-Processor Input issues waves to); the corresponding `RLC_PG_ALWAYS_ON_WGP_MASK` keepalive is set in tandem.

Both writes are PCI-ID-gated to `0x13fe` (BC-250 only) and behave as a no-op on any other AMD GPU; the modified driver behaves identically to the in-tree driver unless the `bc250_cc_write_mode=3` module parameter is set, and the machine reverts to stock 24-CU operation on the next reboot if the parameter is removed from `modprobe.d`. The modification is a patch to the in-tree AMDGPU kernel driver (which AMD ships under GPL-2.0) that programs two GPU configuration registers the stock driver leaves at their fused-off values. It is not a firmware modification: System Management Unit (SMU) firmware, VBIOS, and PSP signed images are not touched, and the patch produces no persistent state on the board.

Independent floating-point sanity check. This paper contributes an independent FP correctness verification of the re-enabled CUs using a Vulkan-compute harness (`bc250-compute-verify.sh` in the project repository [Andrzejczak 2026]). The harness dispatches an FP32 multiply-add kernel over 16 777 216 elements per pass, scheduling

the workgroups so that every Work-Group Processor in the 40-CU configuration receives waves; the device output is compared against a host-computed reference. Three independent passes were run, repeated across two boot cycles, yielding 100 663 296 FP32 elements checked with zero arithmetic errors. The check is sanity-level rather than full silicon validation: it exercises FP32 multiply-add specifically, not the full instruction encoding; it does not stress-test thermals; and it covers one sample board. Within those limits the result is consistent with the upstream report [Duggan 2026]: the factory-disabled silicon on this board is not defective, and the re-enabled CUs participate correctly in GPU compute.

3.5 Inference Runtime Configuration

Ollama runs with the following settings:

- `OLLAMA_VULKAN=1` selects the Vulkan backend.
- `OLLAMA_KV_CACHE_TYPE=q4_0` quantises the KV cache to 4-bit, reducing KV memory by $\approx 4\times$ relative to FP16.
- `OLLAMA_CONTEXT_LENGTH=65536` sets the default context allocation to 64K tokens; benchmark requests override it per call via `num_ctx`.
- `OLLAMA_FLASH_ATTENTION=1` enables flash attention [Dao et al. 2022].
- `OLLAMA_MAX_LOADED_MODELS=1` enforces single-model loading to avoid memory contention.
- `OLLAMA_KEEP_ALIVE=30m` retains the loaded model for 30 minutes; warm-start TTFT is 0.2–2.1 s (Section 5.5).
- `OOMScoreAdjust=-1000` protects the Ollama process from the Linux OOM killer.

A 16 GB NVMe swap file serves as a safety net for memory spikes during model loading. In practice, swap usage was 400–850 MB (OS buffers pushed out by model weights), not active paging.

4 Methodology

The benchmarks are organised into five phases plus a long-context quality sub-study and two follow-up sweeps (40-CU unlock and a reasoning/instruction-tuned coverage extension). Phases 1–5 and the long-context quality runs were performed on the stock 24-CU configuration; the 40-CU unlock (Section 3.4) was characterised in a separate re-run reported in Section 5.7. Clock and power behaviour reported in Section 5.8 was captured from the GPU-overlay sidecar attached to the Phase 1 runs. All measurements were taken on the same hardware (Section 3), with all background services stopped. The original five-phase campaign ran on an earlier stack (Ollama 0.18.0); every per-cell number quoted in the canonical tables of this paper was subsequently re-measured under the co-validated Ollama 0.20.0 / llama.cpp b9265 stack with verified CU mode, while sweep-level distributional summaries over the full 31-model cohort (the context-ceiling distribution of Section 5.3 and the throughput-independent Phase 4 quality battery) retain original-campaign provenance.

Per-pass configuration verification. Every measurement run records its compute-unit configuration at the start from two independent sources (`cat /sys/module/amdgpu/parameters/bc250_cc_write_mode` and `journalctl -k -b 0 | grep active_cu_number`), stored in the environment snapshot released alongside this paper so that the verified-stock 24-CU and patched 40-CU arms cannot be confused at analysis time. The canonical per-cell numbers reported here are $n{=}3$ medians collected under the co-validated stack with the CU mode verified per run. A provenance ledger (`article/data/_provenance_ledger.yaml` in the project repository) maps every quantitative claim in this paper to the data file and verified CU mode from which it derives.

Model identifiers. Models are named by the identifier under which they were served: the Ollama tag form (`name:tag`, e.g. `deepseek-r1:14b`) for the Ollama HTTP backend and the GGUF form (`name-quant`, e.g. `deepseek-r1-14b`) for the direct llama.cpp `llama-completion` path. The same weights can appear under both forms when measured through both runtimes; where that happens, the runtime is stated with the number, since (as Section 5.1 shows) the two paths can differ substantially in throughput.

Needle-retrieval notation. Retrieval results are reported as a fraction whose denominator is stated per table. Where a single marker is planted at a fixed depth, the denominator is the run count (e.g. 2/2, 3/3); where multiple markers are planted in one prompt (the five-needle probe of Section 5.4.3), the denominator is the number of planted markers recovered (e.g. 5/5).

Working-set guard. The harness refuses any cell whose projected working set (model weights plus the Q4_0 KV-cache footprint at the target context) would crowd the 16 GB UMA, recording it as a VM-ceiling skip ("ceil.") rather than letting the run trip the Linux OOM killer. The threshold is a harness parameter (`BC250_VM_CEILING_GIB`) whose default is 11 GiB, the device-local Vulkan heap size with margin. The filled-context sweeps (Tables 5, 7) were run against the full 16 GiB pool with a 14.7 GiB GTT high-water guard, and the Granite state-space ladder (Section 5.4.4) at a 15 GiB threshold, the latter because its per-token KV footprint is far smaller than the dense transformers' and so leaves more of the pool usable for weights. The 40-CU re-run (Section 4.7) used the 11 GiB default for the light models and the same 14.7 GiB guard for the heavy-MoE ladder (Table 12). Each table caption states the value in force.

4.1 Phase 1: Performance Baseline

All 31 models were evaluated with a standardised prompt (a comparative analysis of RISC and CISC architectures), with `num_predict=100` and `num_ctx=4096` (4K context window). The prompt tokenises to 233–324 tokens depending on the model's tokeniser (median 261 across the verified-stock 24-CU perf re-run; range driven by BPE-vs. SentencePiece and special-token handling). Every model in the headline tables has $n{=}3$ runs at this protocol on the verified-stock 24-CU configuration. Recorded metrics: generation speed (tok/s), prefill rate (tok/s), time to first token (TTFT), VRAM consumption, GPU offload percentage, number of GPU layers, and NVMe swap delta.

4.2 Phase 2: Statistical Validation

Eight models spanning the full parameter range (3.2–35 B) were selected for 3-run statistical validation at 4K context. Median, minimum, maximum, and coefficient of variation

(CV) of generation speed were computed. The Phase 3 context-scaling and follow-up cells reported as canonical were collected as $n{=}3$ medians at the 16 GiB pool (Section 4.3); across those 36 filled-context cells (see the provenance ledger [Andrzejczak 2026]) the mean within-cell generation CV is 0.06 % and the maximum 0.37 % (qwen3.5-35b-a3b at filled 64K).

4.3 Phase 3: Filled-Context Scaling

4.3.1 Allocation vs. Utilisation.

This phase addresses a methodological distinction that proved important in practice: the difference between context *allocation* and context *utilisation*. Allocating a large `num_ctx` with a short prompt only tests whether the runtime can reserve KV-cache memory. On this hardware, 128K allocation succeeded consistently; filling 128K with real tokens exceeded the flat 20-minute budget under which this early 128K-utilisation probe was run for larger models (Qwen3.5-35B-A3B MoE, `deepseek-r1:14b`). That probe is separate from, and predates, the canonical filled-context sweep described below, whose per-tier budgets instead scale with context (900–6046 s; see the Filled-Context Protocol) and whose tiers top out at 64K. The two lighter models were directly measured at 128K (`qwen3.5:9b`: TTFT ≈929 s ≈15.5 min, $n{=}2$, needle-recall pass; `granite-4.0-h-tiny`: TTFT ≈147 s ≈2.4 min, $n{=}2$, needle-recall pass; per-tier prefill rates for granite are characterised in Section 5.4.4). Filled-context requests were limited primarily by prefill latency rather than by observed memory exhaustion.

4.3.2 Filled-Context Protocol.

Context was filled to 80 % of `num_ctx` with repeated English text blocks (≈500 tokens each, drawn from diverse technical domains). For each configuration:

1. Target context size: 4K, 8K, 16K, 32K, 64K, 96K, or 128K.
2. Prompt constructed by concatenating text blocks until 80 % fill.
3. The Ollama API's `prompt_eval_count` was verified against the expected token count. A mismatch indicates *silent truncation*: the runtime processed fewer tokens than submitted without error.
4. System RAM and swap monitored via `/proc/meminfo`.
5. Per-tier wall-clock budget enforced, scaled with context size from a 900 s (15 min) floor at the smaller tiers up to ≈6046 s at the largest, sized so that the prefill cost of a high-context cell does not exhaust the budget before the run can complete.

All 31 models were tested; six core models with two runs per configuration, the remainder with a single run (justified by Phase 2 variance results). The canonical cells of Tables 5 and 7 were subsequently upgraded to $n{=}3$ medians by two further independent replicates (Section 4). Ollama silently truncates some models to their native context limit without error, detectable only by checking `prompt_eval_count`. Confirmed cases: `qwen2.5:3b` (caps at 32K) and `phi4:14b` (caps at 16K).

4.4 Phase 4: Competence-Preservation Battery

The Phase 4 battery is explicitly a *competence-preservation* test, not a capability ranking. It is designed to detect broken configurations, quantisation collapse, runtime mishandling of chat templates, and think-token leakage (failure modes that arise when running aggressively quantised LLMs through a non-mainstream runtime stack on non-CUDA silicon) rather than to compare strong models against each other. Each task therefore has a high ceiling: a well-functioning 3–4 B model should pass most of them, and a score below ceiling identifies a stack-level problem rather than a fundamental capability gap. Standard suites (MMLU, HumanEval, GSM8K, perplexity) target capability ranking and are out of scope for this battery; their absence is a deliberate choice, discussed as a scoped limitation in Section 6.

All 31 models were evaluated on five task types, three runs per task per model (465 trials total):

1. **Summarisation.** Given a 120-word passage, produce exactly three sentences. Scored by keyword presence ($\geq$50 %) and sentence count.
2. **JSON extraction.** Given biographical text, return a JSON object with seven specified keys. Scored by valid parse, keys present, and values matching.
3. **Fact recall.** Given five explicit facts, answer a targeted question. Scored by keyword presence.
4. **Instruction following.** Produce exactly five numbered advantages of solar energy. Scored by regex count.
5. **Arithmetic.** Compute 17×23 and return only the number. Scored by substring match for "391".

Scoring was fully automated and deterministic: keyword, JSON-parse, and regex checks executed at test time; no LLM-as-judge or human annotation was used. Complete task prompts and scoring functions are included verbatim in the benchmark script (`bench-rigorous.py`) in the project repository [Andrzejczak 2026]. Models ran with `think: false`, although some model families ignore this directive and consume the output budget with thinking tokens; this is itself a competence-preservation observation.

4.5 Phase 5: Cold-Start Latency

Three models were fully unloaded (`ollama stop`), and the time from request submission to first generated token was measured. Between cold runs the Linux page cache was dropped via `sync && echo 3 > /proc/sys/vm/drop_caches` so that "cold" means cold all the way to NVMe, not warm from the kernel page cache. Three cold runs were collected for each model (Section 5.5).

4.6 Long-Context Quality

To assess whether models can effectively utilise information embedded in large contexts (rather than only generate tokens after a long prompt), two additional test types were run on three models: embedded fact retrieval at 16K, and multi-hop reasoning and synthesis at 16K and 32K:

– **Embedded fact retrieval.** Three unique facts planted at 25 %, 50 %, and 75 % positions; the model must retrieve a specific numerical answer.
– **Multi-hop reasoning.** Four task types requiring either chaining three scattered arithmetic facts or detecting contradictions between studies separated by thousands of filler tokens.

Two independent runs were conducted per model and context size, yielding 48 multi-hop/synthesis trials.

4.7 40-CU Unlock Re-Run

After completing Phases 1–5 and the long-context quality study on the stock 24-CU configuration, the kernel-module modification of Section 3.4 was applied and the short-prompt Phase 1 throughput protocol was re-run on an 11-model subset (selected for parameter-count spread: 9 B–35 B with one high-throughput hybrid-Mamba reference point and the two reasoning/instruction-tuned follow-ups from Section 5.3.4) with three runs per model. KV-cache type, flash-attention setting, prompt template and llama.cpp build are identical between the 24-CU baseline and the 40-CU re-run; only the `bc250_cc_write_mode` modprobe parameter changes between the two passes.

The first filled-context (Step 2) sweep attempt at 40 CUs did not complete: at a high context tier a single `llama` worker grew to $\approx$12 GiB resident against the $\approx$14 GiB usable UMA budget, invoking the Linux OOM killer, which cascaded to a `systemd` process and hung the host. This is a host memory-budget exhaustion from an over-aggressive model $\times$ context tier, not an effect of the unlock: the CU count does not change the 14 GiB UMA budget, so the same tier OOMs identically at 24 CUs. The sweep was subsequently completed after three harness changes that target the failure modes rather than the unlock: (i) a pre-launch VM-ceiling guard that skips any cell whose projected weights + KV working set exceeds 11 GiB instead of letting Linux OOM-kill `init`; (ii) per-cell thermal cooldown gating that idles the board until the GPU edge sensor falls to $\leq$72 °C before each cell, so a measurement is never started from a hot, mid-throttle state; and (iii) operating the board under the `oberon` governor, which on this Cyan Skillfish APU is the only working clock control (the `amdgpu force_performance_level` interface accepts only `auto` and the OverDrive registers reject manual writes) and which caps the core at 1.5 GHz while self-throttling under thermal load. Per-cell GPU telemetry (mean/peak core clock, peak edge temperature, throttle flag) was sampled at 1 Hz throughout so the operating point is recorded rather than assumed; across the completed 40-CU sweep the mean core clock stayed at 1.15–1.47 GHz and the peak edge temperature at 76–95 °C, i.e. entirely within the governor's self-throttling band. The resulting 40-CU filled-context ladder is reported in Section 5.7.

5 Results

5.1 Generation Speed and VRAM (Stock 24-CU)

Table 3 shows results for representative models on the stock 24-CU configuration. Every model achieved 100 % GPU offload after GTT tuning. One additional model (`qwen3:8b` at Q8_0) appears in Section 5.6 as a quantisation comparison but is not counted among the 31 distinct models.

Table 3: Generation speed and VRAM for representative models at 4K context. ⋆ recommended for deployment: `qwen3.5:9b` as the general long-context choice, and `qwen3.5-35b-a3b` as a higher-throughput alternative. All numbers from the verified-stock 24-CU configuration under Ollama 0.20.0, with CU mode confirmed via `journalctl` (`active_cu_number 24`) and the AMDGPU parameter `bc250_cc_write_mode=0`. The 40-CU paired re-run (Section 5.7) is reported separately. Median of $n = 3$ runs; coefficient of variation reported in Table 4.

Model	Par.	Quant	Gen (tok/s)	Pfill (tok/s)	VRAM (GiB)	Qual.
llama3.2:3b	3.2B	Q4_K_M	80.6	384.9	2.2	93 %
phi4-mini	3.8B	Q4_K_M	67.9	357.5	2.5	93 %
gemma3:4b	4B	Q4_K_M	47.8	328.7	3.8	100 %
qwen3:8b	8.2B	Q4_K_M	31.6	161.4	5.1	100 %
⋆ qwen3.5:9b	9.7B	Q4_K_M	23.1	143.1	7.9	100 %
mistral-nemo:12b	12.2B	Q4_0	26.0	130.1	6.7	80 %
qwen3:14b	14.8B	Q4_K_M	20.1	86.4	8.9	100 %
deepseek-r1:14b	14B	Q4_K_M	22.1	82.7	8.5	100 %
⋆ qwen3.5-35b-a3b [a]	35/3B	IQ2_M	25.1	147.3	12.3	93 %

[a] Qwen3.5-35B-A3B: 35 B total, 3 B active per token; IQ2_M ≈2.5 bits/weight.

Three patterns are visible in the data.

MoE on scalar hardware. The Qwen3.5-35B-A3B model with only 3 B active parameters per token reached 25.1 tok/s under Ollama, faster than the 9.7 B dense model (23.1 tok/s) and all 14 B dense models (≈20–22 tok/s) despite its 35 billion total parameters. The one dense model that nominally edges it out on this Ollama path, `mistral-nemo:12b` at 26.0 tok/s, does so at Q4_0 and only 80 % basic-task quality (Table 8), and the controlled same-quantisation comparison below removes the runtime confound entirely. A second MoE in the validation panel, Qwen3-30B-A3B at Q2_K, reaches 41.5 tok/s on the same configuration (Table 4). These are the Ollama-served figures of Table 3; the identical weights and quantisation run ≈2.4× faster through the llama.cpp completion path directly (qwen3.5-35b-a3b: 59.5 tok/s, Table 11). On dense models the two paths differ little (`deepseek-r1:14b`: 22.1 vs 21.2 tok/s, within run-to-run variation, Table 11), so the wrapper penalty is specific to the MoE and MXFP4 expert paths (cf. the ≈1.4× gpt-oss gap, Section 5.6.1) rather than a general property of the Ollama layer. Figure 1 extends the comparison to the llama.cpp backend on the verified-stock 24-CU configuration, where it becomes a controlled test. Held to a single quantisation (IQ2_M) and a single backend, the three Qwen3.x A3B MoE variants (30–35 B total, 3 B active) decode at 58–60 tok/s and the gpt-oss-20b MXFP4 MoE (3.6 B active) at 66 tok/s, whereas `QwQ-32B` [Qwen Team 2025b], a *dense* 32 B model at the *same* IQ2_M quantisation on the *same* llama.cpp build, decodes at only 9.6 tok/s. This ≈6× gap isolates the active-parameter effect: a 35 B-total / 3 B-active MoE decodes at the rate of a small model, not of its 32–35 B dense-equivalent. The dense comparator is reasoning-tuned and not otherwise architecture-matched to the MoEs, so what is held fixed is quantisation, backend and parameter scale rather than every microarchitectural detail; on this bandwidth-bound path (Section 7.2) decode cost is nonetheless set by the weight bytes read per token, which scale with the active count independently of how the model was tuned. The configuration is a measurement device, not a deployment recommendation: against a deployable 14 B dense model the edge is the more modest 14–25 % noted above. The small dense points in Figure 1 (Ollama-served, falling from 80 tok/s at 3 B to 20–26 tok/s at 8–14 B, the 8 B point being the Q8_0 build) trace the same trend from the

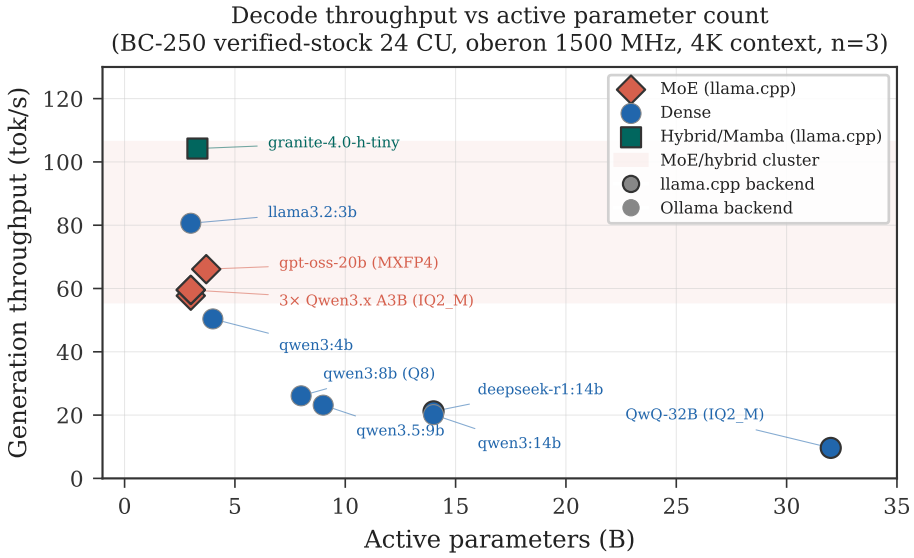


Figure 1: Generation throughput versus active parameter count on the verified-stock 24-CU configuration (oberon 1500 MHz, 4K context, $n = 3$). MoE/hybrid points (orange diamonds, teal square) and the dense `deepseek-r1:14b` and `QwQ-32B` points are benchmarked via llama.cpp (heavier borders); the small dense points (blue circles) via Ollama (lighter borders); the shaded band spans the MoE/hybrid cluster. The ≈6× separation between the Qwen3.x A3B MoEs and the dense `QwQ-32B` at the same IQ2_M quantisation and backend is the controlled comparison of Section 5.1. Point labels note non-default quantisation; weights are Q4_K_M unless marked (MXFP4, Q8, IQ2_M), with total-parameter counts for the sparse models given in the text.

opposite direction as corroboration.

Speed scales inversely with parameters. For dense models at the same quantisation, throughput decreases predictably with parameter count: 3 B ≈68–81 tok/s, 8 B-class ≈26–40 tok/s, 14 B ≈20–22 tok/s. The pattern fits the memory-bandwidth-bounded interpretation discussed in Section 7.2, since each token requires reading all model weights once from GDDR6.

Prefill behaviour. Prefill rates in Table 3 span 83–385 tok/s and track parameter count in the same order as generation: 14 B-class (`qwen3:14b`, `deepseek-r1:14b`) at 83–86 tok/s, the Qwen3.5-35B-A3B MoE at 147 tok/s, the 8–12 B class at 130–161 tok/s, and the 3–4 B class at 329–385 tok/s.

5.2 Statistical Validation

Table 4 shows 3-run validation for eight models. CV remains at or below 0.36 % for all tested models, with the slower dense 12–14 B models holding the tightest spread (`qwen3:14b` and `mistral-nemo:12b` at CV ≤0.04 %), consistent with each token taking longer relative to measurement noise; the two sparse MoEs sit at the loose end of this very narrow band (0.35–0.36 %).

Table 4: Statistical validation: 3 runs per model at 4K context with the short-prompt perf protocol (RISC/CISC prompt, 233–277 tokens across the 8 models in this panel, n_{gen}=100), verified-stock 24-CU configuration, Ollama 0.20.0. This is the short-prompt analogue of Table 5, which uses the filled-context protocol (80 % real-token fill at each tier); absolute throughputs are higher here because there is less KV-cache to traverse per generated token. "Range" is the min–max of the three per-run medians rounded to 0.1 tok/s; CV is the coefficient of variation (sample standard deviation ÷ mean) computed on the unrounded per-run values, so a sub-0.1 tok/s spread can still yield a small non-zero CV.

Model	Median (tok/s)	Range	CV (%)
qwen3:14b	20.1	20.1–20.1	0.04
mistral-nemo:12b	26.0	26.0–26.0	0.01
qwen3:8b	31.6	31.6–31.7	0.14
qwen3.5-35b-a3b	25.1	24.9–25.1	0.36
qwen3.5:9b	23.1	23.1–23.1	0.15
Qwen3-30B-A3B (Q2_K)	41.5	41.4–41.7	0.35
llama3.2:3b	80.6	80.5–80.7	0.12
phi4-mini	67.9	67.8–68.0	0.10

The low within-model variance (CV $\leq$ 0.36 %) is expected for a single-tenant UMA system with no PCIe transfer jitter and all background services stopped, and most likely also reflects the `oberon`-governed GPU clock settling into a single dominant dynamic-power-management (DPM) tier over the duration of a single perf cell (Section 5.8).

5.3 Context Scaling

5.3.1 Generation Speed Under Filled Context.

Table 5 shows how generation speed degrades as context utilisation increases.

Figure 2 summarises the per-model degradation curves. Among models that complete the full 4K-to-64K ladder, dense-model degradation at 64K ranges from $-13\,\%$ (gemma3:4b) to $-76\,\%$ (phi4-mini). qwen3.5:9b sits at $-27\,\%$, mistral-nemo:12b at $-62\,\%$. Two dense models (qwen3:8b, qwen3:14b) exhausted the per-tier wall-clock budget (6046 s) at 64K. The Qwen3.5-35B-A3B MoE completes the full ladder to 64K filled (25.7 tok/s at 4K, 16.6 at 64K, $-35\,\%$), so its active-parameter throughput advantage survives long context. Of the four models that retrieved both needles at 4K, only phi4-mini and qwen3.5:9b retained 3/3 retrieval at 64K; gemma3:4b and mistral-nemo:12b both dropped to 0/3. For gemma3:4b this loss came even though throughput was almost fully preserved ($-13\,\%$ at 64K), showing that attention-quality degradation can be independent of the throughput collapse.

During autoregressive decoding, generating each token requires reading all model weights plus the accumulated KV cache from memory. Because the KV cache grows linearly with context length, per-token generation time $t = 1/\text{throughput}$ is expected to increase roughly linearly with n_{ctx}. The measured data fit this prediction qualitatively: for qwen3.5:9b, t increases by a factor of $1.38\times$ between 4K and 64K (a $16\times$ increase in context); for phi4-mini, by $4.1\times$. The sub-proportional growth on the better-behaved

Table 5: Generation speed (tok/s) and needle-retrieval rate at 80 % real-token context fill, verified-stock 24-CU configuration. Generation tok/s is the median of n=3 independent runs at the 16 GiB pool (`ttm.pages_limit`=4 194 304). "timeout" = run exceeded the per-tier wall-clock budget (900–6046 s, scaled with context); "n/a" = tier above the model's native context window or not run. Needle = X/3 retrievals of two embedded marker codes ("A"- and "Ω"-prefix) at 50 % prompt depth, reported at each model's deepest completing tier; within-cell gen CV $\leq$ 0.4 % in every completed cell. The 4K column is the filled-context (80 %-fill) figure: it tracks the short-prompt Table 3 value for the MoE (qwen3.5-35b-a3b 25.7 vs 25.1, cross-pass variation) but runs lower for the denser small models (phi4-mini 55.0 vs 67.9, qwen3:8b 28.2 vs 31.6) because the 80 %-fill prompt carries more KV cache to traverse per generated token (cf. the short-prompt/filled-context note in Table 4).

Model	4K	16K	32K	64K	Needle@max
⋆ qwen3.5-35b-a3b	25.7	23.3	20.7	16.6	3/3 @ 4K
⋆ qwen3.5:9b	22.6	20.1	18.4	16.4	3/3 @ 64K
gemma3:4b	46.5	45.1	43.4	40.3	0/3 @ 64K
phi4-mini	55.0	34.1	22.6	13.4	3/3 @ 64K
qwen3:8b	28.2	21.0	15.6	timeout	3/3 @ 32K
qwen3:14b	18.7	14.9	11.7	timeout	3/3 @ 32K
mistral-nemo:12b	23.4	17.7	13.3	8.9	0/3 @ 64K
phi4:14b†	19.3	13.9	n/a	n/a	3/3 @ 16K

† phi4:14b's native context window is 16K; tiers above 16K are not applicable.

models indicates that at shorter contexts the fixed cost of reading model weights dominates, and the KV-cache contribution becomes significant only at longer contexts. The underlying memory-bandwidth-bounded interpretation is discussed once in Section 7.2.

5.3.2 Context-Ceiling Distribution.

Of the 31 models tested, 22 (71 %) reach the 64K filled-context ceiling. Five are limited to 32K by native model limits (`qwen2.5:3b`, `qwen2.5:7b`), by impractical latency (`deepseek-r1:14b` becomes generation-bound at filled 64K, dropping to 2.3 tok/s against 21–22 tok/s at 4K; the llama.cpp retrieval probe of Table 10 still recovers all five needles at 64K, so the limit is throughput, not retrieval), or by the 16 GB working-set envelope: the dense `qwen3:8b` and `qwen3:14b` allocate 64K but their 80 %-fill 64K cell times out (Table 5), re-confirmed non-completing even at the full per-tier budget as prefill thrashes against the physical pool. Two are limited by native context windows below 32K: `phi4:14b` (16K, silent truncation) and `gemma2:9b` (8K). One model (`qwen2.5-coder:7b`) produced a `prompt_eval_count` of zero at all fill levels, suggesting a format incompatibility. One model (`qwen3.5-27b`, a ≈13 GiB IQ2_M build) runs to 32K filled context (8.0/7.4/6.8 tok/s at 4K/16K/32K) but not beyond, its weights-plus-KV working set reaching the 16 GB physical envelope at the higher tiers.

5.3.3 KV-Cache Quantisation Impact.

Q4_0 KV-cache quantisation enables large-context inference on 16 GB UMA. On a 14 B reference model (deepseek-r1-14b via llama-completion, n=2) the three KV widths give essentially the same 4K generation throughput (FP16 22.5, Q8_0 21.2, Q4_0 21.1 tok/s;

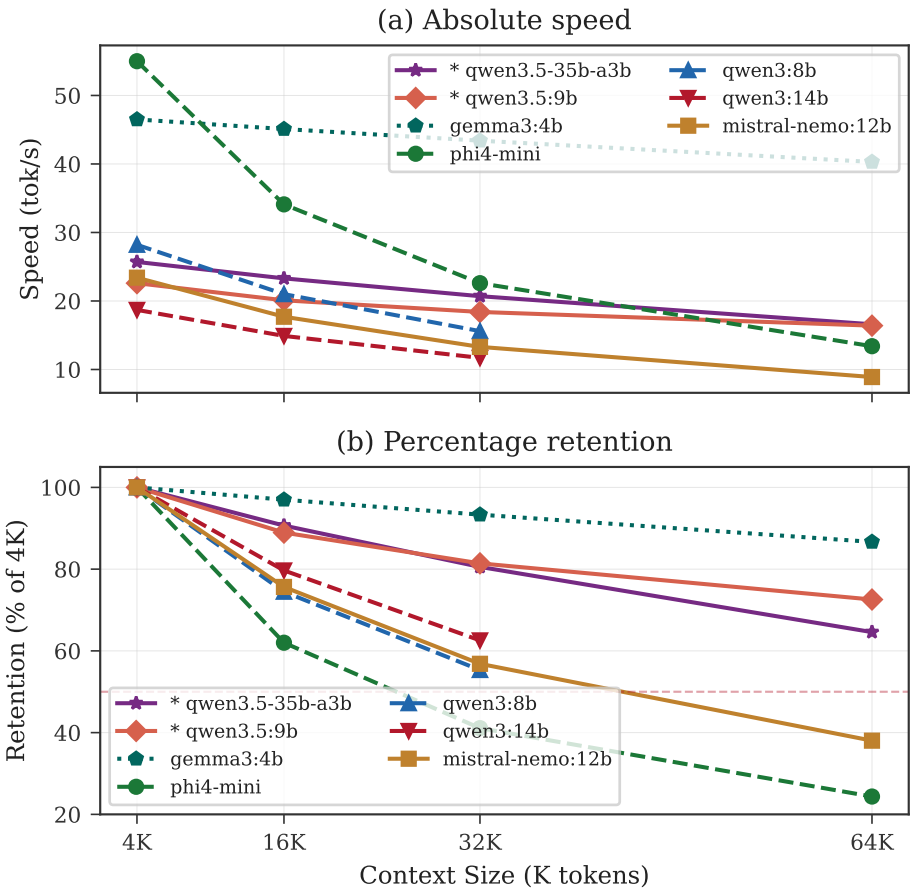


Figure 2: Generation speed degradation with filled context (80 % real-token fill, verified-stock 24-CU, n=3). Panel (a) shows absolute throughput; panel (b) the same data as a fraction of each model's 4K throughput (steepest decay on phi4-mini, flat on gemma3:4b, intermediate on the rest). Per-cell values are listed in Table 5.

FP16 marginally faster, +6 %), confirming that KV access is not the dominant per-token cost at short context (Section 7.2). What the KV width buys is capacity: by the architectural footprint (bytes/token × 24576) the projected working-set ceiling on this model rises from 16–24K (FP16) through 64K+ (Q8_0) to a 128K allocation (Q4_0), so cutting KV bytes ≈4× (FP16→Q4_0) costs no measurable speed yet unlocks the long-context tiers FP16 cannot fit (per-width footprint table in the supplementary material [Andrzejczak 2026]). Table 6 extends the comparison to filled 16K and 32K context on a lighter 9 B model (`qwen3.5:9b` via Ollama, Q4_K_M weights), where Q8_0 fits under the 11 GiB ceiling at both tiers: Q8_0 delivers essentially the same generation speed as Q4_0, differing by +0.3 % at 16K and +0.9 % at 32K, under 1 % at both tiers. On this Vulkan/RADV backend the KV-width setting is a capacity trade-off rather than a speed trade-off: Q4_0 allows 2× more context slots before hitting the working-set

Table 6: KV-cache width effect on long-context generation speed (`qwen3.5:9b` via Ollama, Q4_K_M weights, 24-CU, oberon 1500 MHz, n=3 per cell at the 16 GiB pool). FP16 is omitted (projected working set exceeds 11 GiB at 16K+). The ≤0.9 % Q8_0–Q4_0 difference confirms KV width is a capacity, not a speed, trade-off.

KV Type	Gen @ 16K (tok/s)	Gen @ 32K (tok/s)
Q4_0 (deployed)	20.1	18.4
Q8_0	20.1	18.6

Table 7: Generation speed (tok/s) at filled context for the reasoning- and instruction-tuned follow-up models, verified-stock 24-CU configuration, median of n=3 independent runs at the 16 GiB pool (`ttm.pages_limit`=4 194 304); in-cell gen-CV ≤ 0.5 % across all cells. Both models climb the full 4K–32K ladder within the 16 GiB envelope.

Model	Quant	4K	16K	32K
qwq-32b	IQ2_M	9.3	7.7	6.3
mistral-small-3.2	Q3_K_M	11.1	9.6	8.1

ceiling, with no generation-speed penalty.

5.3.4 Reasoning- and Instruction-Tuned Follow-Up.

The 31-model cohort covered general-purpose chat, code, and MoE classes but did not include a modern reasoning-tuned dense model or a 24–32 B instruction-tuned dense model in the deployable size range. A focused two-model follow-up addressed this: `QwQ-32B` [Qwen Team 2025b] (reasoning-tuned dense, 32 B) at IQ2_M (11.3 GiB) and `Mistral-Small-3.2-24B` [Mistral AI 2025] (instruction-tuned dense) at Q3_K_M (11.5 GiB). Both fit the same ≈11 GiB working-set budget as the existing `qwen3-coder-30b-iq2m` entry and ran the harness with the same Q4_0 KV setting.
Quantisation-induced practical ceiling. A first attempt used the more common deployment quantisations IQ3_M for QwQ-32B (14.8 GiB weights) and Q4_K_M for Mistral-Small-3.2 (14.3 GiB weights). Both files loaded but neither reached usable throughput at 4K: the Q4_K_M Mistral took 5.3 minutes just to read weights from NVMe and prefilled at 0.55 tok/s (vs. 60 tok/s at Q3_K_M); the IQ3_M QwQ timed out before producing a token. The failure mode is weight pages thrashing through swap once weights (≈14 GiB) plus KV plus runtime buffers exceed the 16 GB *physical* pool: at the 16 GiB pool models up to ≈13 GiB of weights run, while the ≈14 GiB Mistral-Small-3.2 Q4_K_M remains non-functional (4K timed out, 0.3 GiB free), swap-bound against the physical RAM rather than against `pages_limit`. Dropping each model by one quantisation step (≈3.5 GiB savings) restores normal operation; the Qwen3.5-35B-A3B MoE clears the same range more comfortably because only the 3 B-active subset is hot per token.

`QwQ-32B` at IQ2_M delivered 9.6 tok/s at 4K on the short-prompt perf protocol (Phase 1, drift CV 0.21 % over $n = 10$ runs) and 9.3 tok/s on the filled-context 4K protocol (Phase 3, structured 80 %-fill prompt; n=3 runs, in-cell CV <0.5 %); `Mistral-Small-3.2` at Q3_K_M delivered 11.1 tok/s on the filled-context 4K protocol (the short-prompt 4K figure for this model in Table 11 is the marginally higher 11.6 tok/s, filled context being slightly slower for the usual KV-traversal reason). Both models held a slow per-token degradation across the full ladder: QwQ drops 33 % from 4K to 32K, Mistral

Table 8: Quality tier distribution across 31 models (5 tasks × 3 runs each). Two models are listed separately: `qwen3.5-27b-iq2m` (functional, with degraded structured output at IQ2_M) and `qwen2.5:7b` (intermittent loading bug).

Score	Models	Representative members
100 %	15	All 14 B, Qwen3 8 B family, gemma3:4b/12b, gemma2:9b, qwen3.5:9b
93 %	5	qwen3.5-35b-a3b, phi4-mini, llama3.2:3b, llama3.1:8b, glm4:9b
80–87 %	3	granite3.3:8b, mistral-nemo:12b, seed-coder-abliterate:8b
73 %	2	qwen2.5:3b, deepseek-r1:8b
≤40 %	4	Think-token leakage or specialised / broken models
53 %	1	qwen3.5-27b-iq2m (functional; valid JSON and 5-item instruction following degraded at IQ2_M)
20 %	1	qwen2.5:7b (intermittent loading bug)

13 % from 4K to 16K and 27 % from 4K to 32K. Both climb to 32K filled context within the 16 GiB envelope (QwQ 6.3 tok/s, Mistral 8.1 tok/s at 32K), the tier at which QwQ's ≈12.7 GiB projected working set (weights plus a Q4_0 KV cache growing at ≈0.14 MiB per allocated token) approaches the practical ceiling. A power-wrapped re-run of both models stayed within the telemetry envelope of Section 5.8 (peak 116 W, T_{max} 76 °C), so adding a 32 B reasoning model to the workload mix did not raise sustained package power on this governor.

Prompt-format dependency. The `Mistral` model emitted an immediate end-of-text token in the `llama-completion` *perf* step (raw-text prompt, no chat-template wrapping), producing valid prefill numbers but no generation tokens; the *ctx-quality* step uses an explicit "Read the document and answer" framing that satisfied the instruction-tuned distribution and produced the data in Table 7. This is a property of strict chat-templated models on raw-text completion and not a runtime defect; the corresponding $n = 2$ perf row is omitted.

5.4 Quality Assessment

5.4.1 Basic-Task Quality.

Table 8 shows quality across all 31 models. Among the 29 in the main distribution (the two separately-listed models excluded), the median score is 100 %: 15 models achieved a perfect 15/15, and every recommended model scored ≥93 %.

Qwen3.5-35B-A3B at IQ2_M (≈2.5 bits/weight) scored 14/15 (93 %); its sole failure was an instruction-following task where it added a preamble before an otherwise correct list. Aggressive quantisation preserved basic-task functionality on this hardware in this configuration.

Failure patterns track model design rather than hardware constraints. Arithmetic ($17 \times 23 = 391$) is the hardest task: five models score 0/3. Fact recall is the easiest: every testable model passes. These are model-level weaknesses rather than BC-250-specific artefacts.

The high median (100 %) is by design of the competence-preservation battery (Section 4.4); tasks where the ceiling effect is weaker (arithmetic and instruction compliance) do separate models that handle structured output from those that do not.

Table 9: Long-context quality: multi-hop reasoning and synthesis at 16K and 32K filled context (48 trials, 300-token output limit). The multi-hop rows are budget-limited: a 1024-token re-run of all three models (n=3 each, 12 trials/model) recovers the two stronger models to 12/12 (`qwen3.5:9b` and the 35 B-A3B MoE) but leaves the 3.8 B `phi4-mini` at 2/12, separating output truncation from a genuine small-model reasoning limit.

Task type	Pass/Tot.	Rate	Pattern
Synth.: timeline	12/12	100 %	Universal pass
Synth.: contradict.	11/12	92 %	Reliable
Multi-hop: popul.	4/12	33 %	Budget-limited
Multi-hop: budget	1/12	8 %	Budget-limited

5.4.2 Long-Context Quality.

Fact retrieval at 16K succeeded without error: 18/18 across three models and three positions (early, middle, late). Multi-hop reasoning proved more challenging (Table 9). Synthesis tasks (contradictions, timelines) were reliable under the original 300-token output budget (23/24 pass). Multi-hop arithmetic, which links three scattered facts, scored far lower (5/24), but under that budget the chain-of-thought was frequently truncated before the final answer. To separate truncation from genuine reasoning failure, all three models were re-run on both multi-hop tasks at a 1024-token budget (n=3 per cell, 16K and 32K; 12 trials per model). The outcome splits cleanly by model scale: the two stronger models recover completely (`qwen3.5:9b` and the 35 B-A3B MoE both 12/12), confirming truncation as the dominant artefact for them, whereas the 3.8 B `phi4-mini` stays weak (2/12), a genuine multi-hop limit at that scale rather than an output-budget effect.

Under the budget-limited 300-token battery the three models scored similarly (qwen3.5-35b-a3b and qwen3.5:9b 10/16, phi4-mini 8/16); the 1024-token re-run resolves this into a clear capability split, the two larger models reaching 12/12 on multi-hop while phi4-mini lags at 2/12. Synthesis-task patterns remain consistent across runs.

5.4.3 Five-Needle Recall at 32K–128K.

A separate retrieval-only probe placed five distinct fixed-content codes (`NEED-N-WORD-WORD-NN`) at depths 10 / 30 / 50 / 70 / 90 % of a filler document, sized so that prefill matches the target context length, and asked the model to enumerate every code seen. Two runs per cell, scored by literal substring match (with whitespace and punctuation normalised). Table 10 summarises the result on four reference models on verified-stock 24-CU at KV q4_0. Three of four models recover all five codes at every tier they reach; `deepseek-r1-14b` clears 32K–64K but the 128K cell ran past its allocated budget without producing output. The `gemma4-latest` 0/5 row matches this community fork's behaviour on every needle probe in this paper: it emits chain-of-thought rather than echoing the planted codes, so the literal-substring scorer records 0 both here and on the single-needle filled-context ladder (0/2 at 40 CU, Table 12, and the same defect at 24 CU). The row is reported but not extrapolated, since the miss is one of output format under a strict scorer rather than a measured throughput failure.

The two models that clear every tier up to 128K (`qwen3.5:9b` and `granite-4.0-h-tiny`) are also the two that pass the filled-context speed sweep at 64K with no measurable

Table 10: Multi-needle long-context retrieval. Five fixed-content codes planted at depths 10/30/50/70/90 % of a filler document; model is asked to enumerate them. Score = number of codes correctly recovered (median across 2 runs). Verified-stock 24-CU, Ollama 0.20.0/llama.cpp b9265, KV q4_0.

Model	32K	64K	128K	Notes
qwen3.59b	5/5	5/5	5/5	all tiers pass
deepseek-r1-14b	5/5	5/5	n/a	128K over budget
granite-4.0-h-tiny	5/5	5/5	5/5	all tiers pass
gemma4-latest	0/5	0/5	0/5	format mismatch†

† `gemma4-latest` emits chain-of-thought instead of the requested code list, so the literal-substring scorer records 0/5; the same output-format effect drives its 0/2 on the single-needle 40-CU ladder (Table 12).

degradation in needle accuracy, supporting their selection as the recommended long-context configurations on this hardware.

5.4.4 Granite-4.0-h-tiny at 128K and 192K Tokens.

A separate ladder targeted the architectural outlier in the panel: the IBM Granite 4.0 h-tiny mamba–MoE hybrid [Granite Team 2025], whose state-space attention path has a much smaller per-token KV-cache footprint than the dense transformers tested elsewhere in this paper. Two runs were attempted at each of three tiers (128K, 192K, 256K tokens), KV q4_0, flash attention on, with a needle planted at the 50 % position and verified in the generated output. Granite passed both runs at both 128K (median generation 74.0 tok/s, prefill 420 tok/s, needle retrieved 2/2) and 192K (65.4 tok/s, prefill 347 tok/s, needle 2/2). The 128K tier here corresponds to an actual prompt of 94 068 granite tokens (the 80 %-fill filler tokenises below the 128K allocation target). At 256K the harness's 15 GiB working-set guard refused the cell: the projected weights-plus-KV footprint (16.7 GiB) exceeds the 16 GiB physical UMA after accounting for kernel and TTM reservations, so the cell is refused before launch as a VM-ceiling skip (`projected_vm_16.7gib_over_15.0gib_ceiling`) rather than burned through a timeout. The practical envelope on this 16 GiB BC-250 is therefore approximately 192K tokens for granite-4.0-h-tiny under flash attention and q4_0 KV; the 256K result is a memory-budget rejection, not an architectural ceiling.

5.5 Cold-Start Latency

On the verified-stock 24-CU configuration, cold-start TTFT (model fully unloaded → first generated token, n=3, each run preceded by a full `ollama stop` + `drop_caches`) is dominated by the NVMe weight load: TTFT is 21.0 s for the Qwen3.5-35B-A3B 11 GiB GGUF, 12.0 s for qwen3.5:9b (6.6 GiB), and 17.1 s for deepseek-r1:14b (8.4 GiB; all three Ollama-served), of which the weight load itself (median `load_ms`) accounts for 20.0 / 11.7 / 16.6 s respectively, an effective read rate of ≈0.55 / 0.56 / 0.51 GiB/s (GGUF size ÷ load time). The three rates cluster tightly, consistent with a single NVMe-bound load path. Dispersion between the first cold run and subsequent cold runs is small (0.4–0.8 s across the three models), consistent with NVMe read-amplification rather than model-side warm-up.

With `OLLAMA_KEEP_ALIVE=30m`, cold start only occurs after 30 minutes of idle time. Warm TTFT is 0.2–2.1 s.

5.6 Quantisation Impact on Speed

Comparing Q4_K_M and Q8_0 on the same model (qwen3:8b) with Q4_0 KV cache (Q4_K_M: 31.6 tok/s gen, 161.4 prefill, 5.1 GiB; Q8_0: 26.1 tok/s gen, 196.3 prefill, 8.5 GiB; both 100 % quality, verified-stock 24-CU): Q8_0 carries a 67 % penalty in weight data volume but is only 17 % slower during generation; on prefill the Q8_0 path is in fact 22 % faster. The non-linear relationship is consistent with Q8_0 requiring simpler dequantisation than the block-scaled Q4_K_M format on a memory-bandwidth-bounded path (Section 7.2).

5.6.1 MXFP4 Path on gpt-oss-20b.

The OpenAI `gpt-oss-20b` model [OpenAI 2025] ships with MXFP4-quantised weights, a non-standard sub-byte format whose Vulkan compute path on RADV is unstable in older llama.cpp builds. An early build in this project's bisect (b8959) failed to load the MXFP4 GGUF on the BC-250 under llama-completion, while the Ollama runtime accepted it via an internal dequantisation fallback. Re-tested on the build used for the rest of this paper (b9265, dedicated A/B pass at 4K context), the MXFP4 GGUF loads natively under llama-completion and runs at 61.6 tok/s under the dedicated probe (66.1 tok/s on the perf step, three runs, CV 0.85 %); the Ollama-served variant of the same model on the same box runs at 46.7 tok/s under the probe (45.9 tok/s on the perf step, three runs, CV 0.11 %). The ≈32–44 % advantage of the llama-completion path, on a model where the runtime difference would otherwise be expected to be small, indicates that the Ollama variant in this build is taking a less direct kernel path on the BC-250 (likely on-the-fly dequantisation to a wider intermediate type). The practical reading is that on this driver MXFP4 weights are better served through llama.cpp directly than through the Ollama wrapper when generation throughput matters.

5.7 40-CU Unlock Performance Impact

Table 11 reports the short-prompt throughput delta on the 11-model paired re-run subset. Median of three runs per cell; identical prompt, KV-cache type, flash-attention setting and runtime build (llama.cpp b9265, Ollama 0.20.0) between the two columns; the 24-CU and 40-CU arms were measured on the same board with a full reboot between arms to reset the AMDGPU governor and VRAM state.

Two regularities are visible in Figure 3. First, the speed-up is larger for prefill (median 1.50×) than for generation (median 1.32×): adding compute resources without adding memory bandwidth helps prefill (compute-heavy parallel matrix work) proportionally more than single-token decode, which is dominated by weight reads from GDDR6 (Section 7.2). Second, the smallest generation speed-ups (1.12× on `gpt-oss-20b-ollama`, 1.14× on `gemma4-latest`, 1.24× on the hybrid-Mamba `granite-4.0-h-tiny`) belong to models whose 24-CU throughput is already high relative to their parameter count, compatible with the same memory-bandwidth-bounded picture in which adding ALUs helps least, while the larger speed-ups (1.43–1.57×) belong to models where the 24-CU configuration appears to leave compute headroom. This bandwidth-bounded interpretation is corroborated by the roofline measurement in Section 7.2 (ridge 10.9 FLOP/byte; decode arithmetic intensity ≤ 6.4 FLOP/byte for all tested quantisations). The upstream report [Duggan 2026] reports a 1.61× prefill (pp512) speed-up at a slightly different operating point (1.5 GHz core, ≈125 W board power); the 1.50× median reported here is in the same regime.

Table 11: Throughput before and after the 40-CU unlock on the 11-model paired re-run subset. All numbers in tok/s, median of 3 runs at 4K context, prompt 235–298 tokens depending on tokeniser (median 252), KV q4_0, flash attention on, llama.cpp b9265, with the clock governor held fixed within each model's 24-vs-40 pair. Δ columns give the multiplicative speed-up from verified-stock 24 CUs to patched 40 CUs. All eleven generation deltas and all eleven prefill deltas are positive ($p < 0.01$, sign test, two-sided).

Model	Gen 24	Gen 40	Δ_{gen}	Pfill 24	Pfill 40	Δ_{pf}
qwen3.59b (Ollama)	23.1	31.1	1.34×	143.1	225.3	1.57×
gemma4-latest	28.9	33.0	1.14×	304.0	463.5	1.52×
deepseek-r1-14b	21.2	30.4	1.43×	98.0	140.5	1.43×
gpt-oss-20b (MXFP4)	66.1	87.5	1.32×	181.0	248.3	1.37×
gpt-oss-20b (Ollama)	45.9	51.4	1.12×	253.6	390.4	1.54×
qwen3-coder-30b (IQ2_M)	57.7	76.7	1.33×	178.2	264.6	1.49×
qwen3.5-35b-a3b (IQ2_M)	59.5	78.7	1.32×	171.8	245.9	1.43×
qwen3.6-35b-a3b (IQ2_M)	59.6	78.0	1.31×	170.8	250.2	1.46×
granite-4.0-h-tiny (hybrid)	104.2	129.2	1.24×	510.6	765.0	1.50×
mistral-small-3.2 (Ollama)	11.6	18.3	1.57×	51.5	83.7	1.62×
qwq-32b (IQ2_M)	9.6	14.9	1.55×	47.9	74.8	1.56×
Median across set			**1.32×**			**1.50×**

Filled-context behaviour after the unlock. The completed 40-CU filled-context sweep (Section 4.7) extends the unlock characterisation from the short-prompt operating point to the long-context regime (Table 12). The headline result is that all four heavy MoE builds, run under the 14.7 GiB working-set guard, climb the full 4K–64K ladder at 40 CUs with perfect needle retrieval (2/2 at every tier): the flagship `qwen3.5-35b-a3b` holds 74.1→47.8 tok/s across the span, `qwen3.6-35b-a3b` 73.6→47.9, `qwen3-coder-30b` 65.5→21.8, and the `gpt-oss-20b` MXFP4 MoE 86.0→57.7. The sparse-MoE throughput advantage therefore survives the long-context regime *under the unlock*, not only at stock 24 CUs (Section 5.3). The lighter models behave as at 24 CUs: the hybrid-Mamba `granite-4.0-h-tiny` holds 126→110 tok/s (87 % retention) and `qwen3.5:9b` 31.3→23.4 tok/s (75 %), both at 2/2 needle every tier; `gemma4-latest` keeps healthy throughput (30.9→24.0) but retrieves the needle at no tier, reproducing its 24-CU needle-recall defect (Section 5.4.3); and the dense `deepseek-r1-14b` runs to 24K before the VM guard, the same memory-bound ceiling it hits at 24 CUs. Prefill degrades smoothly with depth at the unlocked operating point (`qwen3.5-35b-a3b` 381→225 tok/s, `granite-4.0-h-tiny` 1080→832 tok/s over 4K–64K), confirming the unlock introduces no long-context cliff: the scaling shape matches the 24-CU curves, shifted up by the CU speed-up.

A 24-CU companion ladder, run under the same `oberon` configuration, gives the per-tier generation speed-up Δ_{gen}. For the four heavy MoEs it grows monotonically with context depth, from a per-tier median 1.32× at 4K to 1.43× at 64K (overall median 1.38× across the full MoE ladder), with `qwen3-coder-30b` steepest at 1.34→1.55×, consistent with the rising share of KV-cache attention computation and with these MoEs leaving more compute headroom at 24 CUs than the bandwidth-bound hybrids. The lighter models follow the same shape at a lower level: `granite-4.0-h-tiny` and `qwen3.5:9b` 1.23×/1.35× at 4K rising to 1.27×/1.43× at 64K, `gemma4-latest` 1.14–1.28× and `deepseek-r1-14b` 1.48–1.55× before the VM ceiling; their filled median is 1.27×, close to the short-prompt cohort median of Table 11 (1.32×). The prefill speed-up is stable at 1.59× (range 1.56–1.63× per model). All confirm that the unlock's benefit carries over to filled context for every model class rather than washing out as the KV

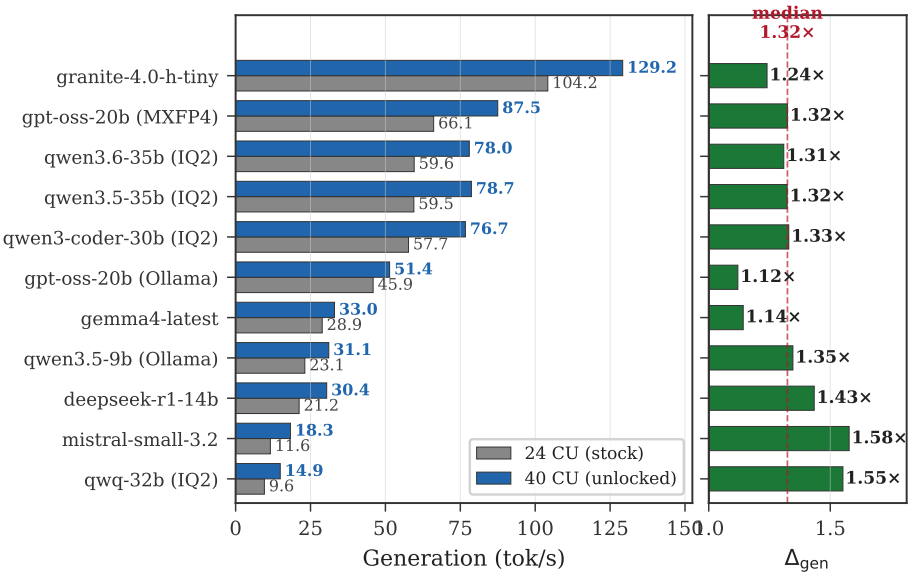


Figure 3: 40-CU unlock A/B on the 11-model paired re-run subset at 4K context. Left: generation throughput before (24 CU, grey) and after (40 CU, blue) the unlock; right: per-model speed-up factor Δ_{gen} with the cohort median (1.32×) marked. Same prompt, KV q4_0 and llama.cpp build between arms. Per-model values are listed in Table 11; the qualitative ranking places the largest gains on compute-bound models and the smallest on bandwidth-bound MoE and Mamba/hybrid models such as `gpt-oss-20b-ollama` and `granite-4.0-h-tiny`.

cache grows. These deltas are, if anything, conservative: the two arms share the `oberon` configuration but not the realised clock. Per-cell telemetry shows the 40-CU arm drawing more power (median 116 vs 101 W per active cell) and running hotter (peak edge 87 vs 82 °C), so the self-throttling governor holds it at a ≈3–4 % lower mean clock (1.41 vs 1.46 GHz); normalising each cell by its clock raises the median Δ_{gen} from 1.27× to ≈1.31×. The unlock is thus bounded by the board's power and cooling envelope, not only the memory-bandwidth wall of Section 7.2, and a better-cooled unit would likely gain more. The per-tier Δ_{gen} table is in the supplementary material [Andrzejczak 2026].

5.8 Clock and Thermal Behaviour Under Load

GPU clock, edge temperature and package power were sampled from the AMDGPU `sysfs`/`hwmon` interfaces alongside the verified-stock 24-CU perf runs: a 1 Hz overlay on every perf cell, a dedicated 5 Hz sustained-inference sweep over five reference models (612 s, 3 063 samples), and a 10 Hz reconciliation pass (2 314 samples). Two facts matter for reproducibility on this platform.

First, under inference the `oberon`-governor userspace daemon caps the core at the **1 500 MHz** DPM tier, not the firmware-reported 2 000 MHz ceiling: all three sampling rates agree on the 1 500 MHz maximum, and the time-weighted mean sclk sits

Table 12: 40-CU filled-context ladder: generation speed (tok/s) and needle retrieval at 80 % real-token fill. Top group: the original re-run models under the 11 GiB VM ceiling. Bottom group (below the rule): the four heavy MoE builds, run under the 14.7 GiB working-set guard so they are no longer excluded; all four climb the full 4K–64K ladder with 2/2 needle at every tier (cf. their 24-CU ladder, which differs only by the CU speed-up). Median of n=2 runs, KV q4_0, llama.cpp b9265, board under the `oberon` 1.5 GHz-capped governor (measured peak edge 76–95 °C). "ceil." marks the harness VM-ceiling skip. Needle is X/2 retrievals of an embedded marker token at 50 % prompt depth; `gemma4-latest`'s 0/2 reproduces its 24-CU needle-recall failure (chain-of-thought output under the strict scorer, Section 5.4.3), not a throughput failure.

Model	4K	16K	32K	64K	Needle@max
⋆ granite-4.0-h-tiny	126.0	122.6	115.5	110.0	2/2 @ 64K
⋆ qwen3.5:9b	31.3	29.2	25.6	23.4	2/2 @ 64K
gemma4-latest	30.9	30.1	27.2	24.0	0/2 @ 64K
deepseek-r1-14b	27.4	20.2	ceil.	ceil.	2/2 @ 24K‡
Heavy MoE builds (14.7 GiB working-set guard):					
⋆ qwen3.5-35b-a3b	74.1	67.7	59.7	47.8	2/2 @ 64K
qwen3.6-35b-a3b	73.6	67.2	59.2	47.9	2/2 @ 64K
qwen3-coder-30b	65.5	46.6	33.9	21.8	2/2 @ 64K
gpt-oss-20b (MXFP4)	86.0	78.2	69.8	57.7	2/2 @ 64K

‡ `deepseek-r1-14b`'s deepest completed tier is the sweep's 24K intermediate tier (not shown as a column; gen 17.6 tok/s, needle 2/2), VM-ceiling-skipped at 32K. The shallower 16K cell recovered only 1/2 across the two runs, run-to-run variance from this reasoning model occasionally exhausting the 200-token budget before emitting the marker.

at 1.05–1.19 GHz as the clock cycles between the 1 000 and 1 500 MHz tiers (compute bursts during prefill interleaved with memory-bound decode). Second, the clock cannot be pinned from userspace on this platform: the SMU is locked, the kernel `power_dpm_force_performance_level` interface accepts only `auto` (a `manual` tier pin returns `EINVAL`), and the `oberon`-governor is the only working clock control, so the reported throughput reflects the `oberon`-capped operating point under load rather than a manually pinned reference point. The governor was configured identically for every run reported here (`/etc/oberon-config.yaml`: core 1000–1500 MHz, 900–1000 mV). Absolute BC-250 throughput should therefore be quoted together with the observed sclk distribution; the upstream 40-CU report [Duggan 2026] characterises the unlock at 1.5 GHz / ≈125 W / 83 °C, consistent with the `oberon`-capped 1.5 GHz operating point reported here.

Edge temperature stayed in a 57–75 °C band during the sustained sweep (peak 75 °C; per-model perf-run maxima 70–74 °C), well below the 95 °C junction limit, and SMU-reported package power peaked at 119 W (mean 52 W), agreeing with the upstream 125 W figure. These power numbers come from the uncalibrated on-die SMU sensor (`power1_average`); AMD publishes no accuracy specification for it on Cyan Skillfish and no wall-meter cross-check was performed, so they are reported as operating-regime telemetry only: no joules-per-token, tokens-per-joule, or perf-per-watt claim is made in this work. The full per-model clock table, the telemetry traces, a cross-pass determinism check (≤0.13 % drift across 45-minute-spaced passes), a concurrent-inference contention probe, a controlled `GGML_VK_PREFER_HOST_MEMORY` heap-placement A/B, and the raw 5 Hz/10 Hz sampler CSVs are provided as supplementary material in the project repository [Andrzejczak 2026].

6 Threats to Validity

This section consolidates the bounds on what can be concluded from the measurements above, grouped by the dimension along which they bite.

Single device, single driver, single firmware. All measurements come from one BC-250 board running Mesa RADV Vulkan 1.4.328 on Fedora 43 (kernel 6.18.9-200.fc43.x86_64), with canonical numbers from Ollama 0.20.0 / llama.cpp b9265. Harvest-mask variation between individual BC-250 units, thermal solutions (the BC-250 was shipped with diverse passive coolers across mining-rig manufacturers), and kernel/Mesa version drift could all shift the observed numbers. External validation on other GFX1013 hardware would tighten two findings in particular: the CU-unlock speed-up (Section 5.7) and the clock and thermal characterisation (Section 5.8). No comparison with other GPUs, NPUs, or cloud inference is provided; cross-platform comparison is out of scope.

Standard LLM benchmarks not run. MMLU, HumanEval, GSM8K, and perplexity were not executed: the Phase 4 battery (Section 4.4) detects broken configurations rather than ranking capability, so this paper does not contribute to model leaderboards. Practitioners who need capability ranking on this hardware can run the standard suites against the same runtime configuration using the scripts in the project repository [Andrzejczak 2026].

Multi-hop quality under a constrained output budget. The multi-hop scores in Table 9 were collected under a 300-token output limit that truncated chain-of-thought reasoning before the answer. A 1024-token re-run of all three models (n=3, 16K and 32K) resolves the budget artefact: the two stronger models recover to 12/12 (`qwen3.5:9b` and the 35 B-A3B MoE), while `phi4-mini` stays at 2/12. The residual phi4-mini weakness is therefore a genuine multi-hop limit at the 3.8 B scale, not a measurement threat.

40-CU re-run scope. The 40-CU unlock was characterised on an 11-model short-prompt subset (Table 11) and, at long context, on the light re-run models plus the four heavy MoE builds run under the 14.7 GiB working-set guard (Table 12); a full 31-model 40-CU re-run was not performed. The two 40-CU and 24-CU arms are not clock-matched, making the reported Δ_{gen} a conservative estimate of the pure compute effect (Section 5.7).

Telemetry calibration. Power figures throughout Section 5.8 come from the on-die AMDGPU SMU sensor, for which AMD publishes no accuracy specification on Cyan Skillfish and no wall-meter cross-check was performed. Joules per token, perf-per-watt and similar efficiency numbers are therefore not reported; absolute energy claims would require a calibrated PMBus shunt on the 12 V input.

7 Discussion

7.1 The Allocation–Utilisation Gap

As established in Section 4.3, 128K *allocation* succeeds on this 16 GB UMA while 128K *utilisation* is prefill-bound. The platform-independent takeaway: any Ollama-based

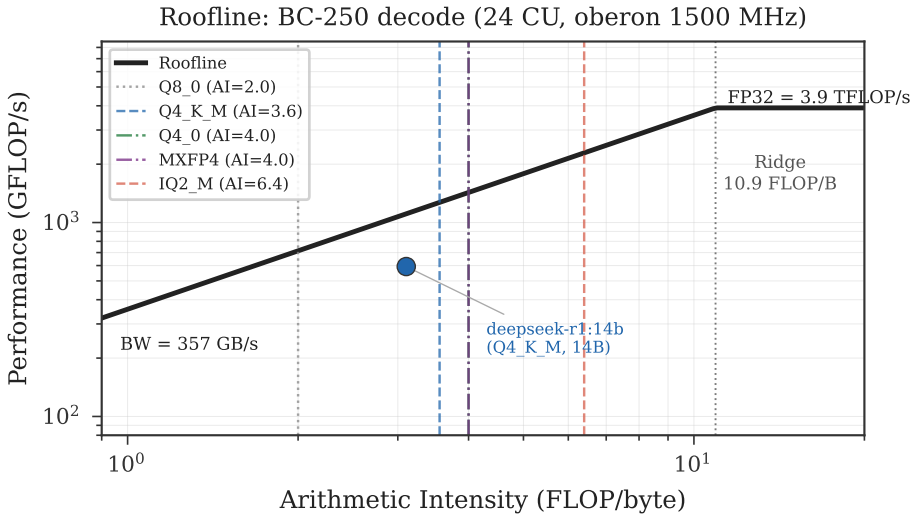


Figure 4: Roofline plot for the BC-250 at 24 CU and 1500 MHz oberon cap. The solid line is the hardware roofline obtained from two Vulkan microbenchmarks: 357 GB/s peak streaming bandwidth (streaming copy) and 3901 GFLOP/s peak FP32 throughput (parallel FMA chains), meeting at a ridge of 10.9 FLOP/byte. Vertical dashed lines mark the arithmetic intensity of each quantisation format during single-token decode ($AI = 2/\text{bytes per weight}$). All tested formats sit left of the ridge, indicating that decode is limited by weight reads rather than by available compute (bandwidth-bound relative to compute). The plotted data point is `deepseek-r1-14b` (Q4_K_M, 14 B dense, llama-completion) on verified-stock 24-CU at 21.2 tok/s, which achieves 593 GFLOP/s and an effective 191 GB/s, ≈53 % of the streaming bandwidth peak, so a single decode stream does not fully saturate the bus even though it sits far left of the compute ridge.

filled-context benchmark must verify `prompt_eval_count` against the expected token count, since Ollama silently truncates to a model's native limit and a headline "128K context" figure can otherwise correspond to a much shorter cropped prompt.

7.2 Memory-Bandwidth-Bounded Decode on Scalar Hardware

Several patterns reported in Section 5 are consistent with one mechanism: throughput scaling inversely with parameter count, per-token generation time growing linearly with context, the non-linear Q4_K_M/Q8_0 quantisation gap, and the larger prefill- than generation-speed-up under the CU unlock. In each case the binding constraint is the same: on a GPU lacking dedicated matrix accelerators, single-token autoregressive decode is dominated by the cost of reading model weights once per token from GDDR6; adding ALUs (the CU unlock) therefore helps prefill proportionally more than generation (median 1.50× vs 1.32×), while reducing weight bytes per token (smaller models, smaller quantisation, sparse activation in MoE) translates almost directly into throughput.

A roofline analysis (Figure 4) provides direct experimental support. Two Vulkan compute microbenchmarks executed on the 24-CU board under the oberon 1500 MHz cap mea-

sured a peak streaming bandwidth of **357 GB/s** (streaming copy of 512 MiB host-coherent buffers in the UMA address space) and a peak FP32 throughput of **3901 GFLOP/s** (eight independent `fma` chains per thread), giving a ridge point of **10.9 FLOP/byte**. All LLM decode quantisations span 2.0 FLOP/byte (Q8_0) to 6.4 FLOP/byte (IQ2_M), each well below the ridge, placing autoregressive decode left of the compute ridge for every quantisation format tested, i.e. in the regime where throughput is limited by weight reads rather than by available compute. Taking the FP32 fused multiply-add (FMA) rate as the compute ceiling is conservative for this purpose: the dequantise-then-multiply path that decode actually executes adds unpacking work per weight and cannot exceed the FP32 throughput, so the true compute ceiling for quantised decode is no higher than the 3901 GFLOP/s plotted and the left-of-ridge placement only strengthens. As a concrete cross-check, `deepseek-r1-14b` (Q4_K_M, 8.4 GiB GGUF, 21.2 tok/s via llama-completion on verified-stock 24-CU) achieves an effective 191 GB/s, ≈53 % of the streaming peak, at an arithmetic intensity of ≈3.1 FLOP/byte (plotted in Figure 4), far left of the 10.9 FLOP/byte ridge. Decode is therefore bandwidth-bound *relative to compute*: far below the compute ceiling, so adding ALUs (the CU unlock) does little for it while reducing weight bytes per token helps almost directly. This is not the stronger claim that decode is bandwidth-*saturated*: at only ≈53 % of the streaming peak the device-level limit is more plausibly occupancy or memory latency than raw bandwidth. The roofline thus fixes which side of the ridge decode sits on (not that the bus is full), and anchors the memory-bandwidth-bounded reading earlier sections refer to.

8 Conclusions

This paper used the AMD BC-250, a repurposed cryptocurrency-mining APU with 16 GB unified GDDR6 and a Vulkan-only GPU compute path, as a single-board testbed. Three transferable findings and the enabling infrastructure that supports them, in the order of Section 1:

1. **Silent prompt truncation in Ollama** is a platform-independent benchmark-validity threat: the runtime can crop prompts above a model's native window, returning HTTP 200 with a reduced `prompt_eval_count` and no error, so any filled-context benchmark must verify that field (Sections 4.3, 7.1).

2. **Allocation ≠ utilisation:** 128K allocation succeeds for every model tested, but filling 128K is prefill-bound and timed out for the larger models while lighter ones completed (qwen3.5:9b, granite-4.0-h-tiny; TTFT ≈15.5 and 2.4 min); both should be reported (Section 4.3).

3. **Sparse MoE on scalar hardware tracks active-parameter count:** Qwen3.5-35B-A3B reaches 25.1 tok/s at 4K (ahead of the 14 B dense models at 20–22), fits 35 B parameters in 16 GB at IQ2_M with 93 % basic-task accuracy, and runs to 64K filled context, so the advantage holds across the context range (Section 5.3), following the memory-bandwidth-bounded picture of Section 7.2. The controlled ≈6× same-quantisation comparison that isolates the active-parameter effect is reported in Section 5.1.

4. **Enabling infrastructure:** raising the in-tree TTM `pages_limit` to 16 GiB is required before the >7 B-parameter class runs at all (the legacy `amdgpu.gttsize`

knob is deprecated on modern kernels), and the community 40-CU unlock [Duggan 2026], here FP-sanity-checked ($\approx 10^8$ error-free FP32 multiply-adds) and re-run paired-arm, yields a median 1.50× prefill and 1.32× generation speed-up, so BC-250 throughput is ambiguous unless the CU mode is stated (Sections 3.4, 5.7).

Repurposed UMA APUs are usable for edge-AI inference once the kernel memory subsystem (`ttm.pages_limit`) is configured and, where the hardware permits, the factory CU-harvest mask is undone.

Acknowledgements

Generative-AI use. The author discloses that AI-assisted tools, including Anthropic's Claude (Opus and Sonnet generations), were used for code prototyping, programming support, and limited editorial assistance during manuscript preparation. All such outputs were reviewed and revised by the author, who approved the final version of the manuscript and takes full responsibility for its content. No generative-AI tool is listed as a co-author.
Author contribution. AA: Conceptualisation, Methodology, Software, Investigation, Data curation, Formal analysis, Writing – original draft, Writing – review and editing, and Visualisation. The 40-CU unlock characterised here was originally proposed and implemented by S. Duggan (cited in Section 3.4). The heap-placement A/B (supplementary material [Andrzejczak 2026]) was prompted by an issue-tracker contribution [BC-250 Issue 4 2026] from Daniel Hoffman (GitHub "thehoff"), whose Vulkan heap analysis motivated the controlled re-run. The author thanks both contributors, without whose public work this paper would not exist in its current form.
Competing interests. The author declares no competing interests.
Data availability. Benchmark scripts, raw result files, configuration files and analysis code are available in the project repository [Andrzejczak 2026]. The CU-unlock build scripts and FP-verification harness used in this work derive from the upstream community repository [Duggan 2026].

References

[Abdin et al. 2024] Abdin, M., Aneja, J., Behl, H., et al.: "Phi-4 Technical Report"; arXiv preprint arXiv:2412.08905 (2024).

[Andrzejczak 2026] Andrzejczak, A.: "BC-250 Benchmark Scripts and Results"; GitHub repository, `akandr/bc250` (2026), https://github.com/akandr/bc250.

[BC-250 Docs 2026] AMD BC-250 Community Documentation Project: "AMD BC-250 Documentation"; community-maintained wiki (2026), https://elektricm.github.io/amd-bc250-docs/ (accessed 10 June 2026).

[BC-250 Issue 4 2026] Hoffman, D. (thehoff) and Andrzejczak, A.: "`GGML_VK_PREFER_HOST_MEMORY=1`: Vulkan heap-placement effect on BC-250 throughput"; Issue #4, GitHub repository `akandr/bc250` (2026), https://github.com/akandr/bc250/issues/4 (accessed 29 May 2026).

[Benazir and Lin 2025] Benazir, A., Lin, F. X.: "Profiling Large Language Model Inference on Apple Silicon: A Quantization Perspective"; arXiv preprint arXiv:2508.08531 (2025), https://doi.org/10.48550/arXiv.2508.08531.

[Chen and Ran 2019] Chen, J., Ran, X.: "Deep Learning with Edge Computing: A Review"; Proceedings of the IEEE, 107, 8 (2019), 1655–1674, https://doi.org/10.1109/JPROC.2019.2921977.

[Dao et al. 2022] Dao, T., Fu, D. Y., Ermon, S., Rudra, A., Ré, C.: "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness"; Proc. NeurIPS, New Orleans (2022), arXiv:2205.14135.

[DeepSeek-AI 2025] DeepSeek-AI: "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning"; Nature, 645 (2025), 633–638, arXiv:2501.12948.

[Duggan 2026] Duggan, S.: "Re-enabling Fused-Off Compute Units on the AMD BC-250 APU via Register-Level Modification"; whitepaper `docs/whitepaper-cu-unlock.pdf`, GitHub repository `duggasco/bc250-40cu-unlock`, commit `0e7b403` (2026), https://github.com/duggasco/bc250-40cu-unlock/blob/main/docs/whitepaper-cu-unlock.pdf (accessed 29 May 2026).

[Fedus et al. 2022] Fedus, W., Zoph, B., Shazeer, N.: "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity"; Journal of Machine Learning Research, 23, 120 (2022), 1–39, arXiv:2101.03961.

[Frantar et al. 2023] Frantar, E., Ashkboos, S., Hoefler, T., Alistarh, D.: "GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers"; Proc. ICLR, Kigali (2023), arXiv:2210.17323.

[Gemma Team 2024] Gemma Team: "Gemma 2: Improving Open Language Models at a Practical Size"; arXiv preprint arXiv:2408.00118 (2024).

[Gemma Team 2025] Gemma Team: "Gemma 3"; model card, Hugging Face (2025), https://huggingface.co/google/gemma-3-4b-it (accessed 7 June 2026).

[ggml-org 2026] ggml-org: "llama.cpp: LLM Inference in C/C++"; open-source project, GitHub repository `ggml-org/llama.cpp` (2026), https://github.com/ggml-org/llama.cpp.

[Granite Team 2025] Granite Team, IBM: "Granite 4.0 Language Models"; model card, Hugging Face (2025), https://huggingface.co/ibm-granite/granite-4.0-h-tiny (accessed 7 June 2026).

[Grattafiori et al. 2024] Grattafiori, A., Dubey, A., Jauhri, A., et al.: "The Llama 3 Herd of Models"; arXiv preprint arXiv:2407.21783 (2024).

[Kernel AMDGPU Params 2026] Linux Kernel Documentation: "Module Parameters: `amdgpu`"; (2026), https://docs.kernel.org/gpu/amdgpu/module-parameters.html.

[Kernel TTM 2025] Linux Kernel Source: "TTM Pool Allocator: `pages_limit` Module Parameter"; Linux v6.15, `drivers/gpu/drm/ttm/ttm_pool.c` (2025), https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/drivers/gpu/drm/ttm/ttm_pool.c?h=v6.15.

[Larabel 2025] Larabel, M.: "RADV Vulkan Driver Support Being Worked On For The GPU Found In The Sony PS5 & BC-250"; Phoronix (2025), https://www.phoronix.com/news/AMD-RADV-PS5-BC-250.

[Lin et al. 2024] Lin, J., Tang, J., Tang, H., et al.: "AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration"; Proc. MLSys, Santa Clara (2024), arXiv:2306.00978.

[LLVM 2026] LLVM Project: "AMDGPU Processor Table"; LLVM documentation (2026), https://llvm.org/docs/AMDGPUUsage.html#processors.

[Mesa 2026] Mesa 3D Graphics Library: "RADV Vulkan Driver Documentation"; (2026), https://docs.mesa3d.org/drivers/radv.html.

[Mistral AI 2025] Mistral AI: "Mistral Small 3.2 24B Instruct"; model card (2025), https://huggingface.co/mistralai/Mistral-Small-3.2-24B-Instruct-2506 (accessed 10 June 2026).

[Ollama 2026] Ollama: "Ollama: Run LLMs Locally"; open-source project, GitHub repository `ollama/ollama` (2026), https://github.com/ollama/ollama.

[OpenAI 2025] OpenAI: "gpt-oss-20b"; model card, Hugging Face (2025), https://huggingface.co/openai/gpt-oss-20b (accessed 7 June 2026).

[Pope et al. 2023] Pope, R., Douglas, S., Chowdhery, A., et al.: "Efficiently Scaling Transformer Inference"; Proc. MLSys, Miami Beach (2023), arXiv:2211.05102.

[Qwen Team 2024] Qwen Team: "Qwen2.5 Technical Report"; arXiv preprint arXiv:2412.15115 (2024).

[Qwen Team 2025a] Qwen Team: "Qwen3 Technical Report"; arXiv preprint arXiv:2505.09388 (2025).

[Qwen Team 2025b] Qwen Team: "QwQ-32B: Embracing the Power of Reinforcement Learning"; vendor blog post (2025), https://qwenlm.github.io/blog/qwq-32b/ (accessed 10 June 2026).

[Qwen Team 2026a] Qwen Team: "Qwen3.5: Towards Native Multimodal Agents"; vendor blog post (2026), https://qwen.ai/blog?id=qwen3.5 (accessed 10 June 2026).

[Qwen Team 2026b] Qwen Team: “Qwen3.6-35B-A3B”; model card, Hugging Face (2026), https://huggingface.co/Qwen/Qwen3.6-35B-A3B (accessed 7 June 2026).
[Zhou et al. 2024] Zhou, Z., Ning, X., Hong, K., et al.: “A Survey on Efficient Inference for Large Language Models”; arXiv preprint arXiv:2404.14294 (2024).